OPERATING SYSTEMS – ASSIGNMENT 1

Name: S. Sharvesh **Register No:** 192525458

TASK 5 – Examine System Calls (4 Marks)

Examine system-call categories: process control, file management, device management, information maintenance, communication and protection.

```
Explain step by step what happens when a program calls read(): library call, trap, mode switch, kernel work and return; how
```

Alternative Concept and Complexity Analysis: If programs use fopen, fread and fwrite instead of open, read and write directly, evaluate system-call count for n bytes and extra library buffer space.

Evidence Expected: A step-by-step system-call explanation with strace evidence from a real program.

ANSWER

5.1 Categories and hospital examples

```
5.2 What happens during read()

The application calls the C library wrapper read(fd, buffer, count).
```

The wrapper places the system-call number and arguments in the ABI-prescribed registers or memory locations.

The wrapper executes a trap/syscall instruction, transferring control from user mode to kernel mode.

The CPU switches to the kernel entry point and the kernel validates the file descriptor, user buffer address and requested access.

The kernel obtains data from the filesystem, page cache or device. If data is not ready, the process may block and another process may run.

The kernel copies or maps the data to the user buffer, updates file position and returns a byte count or error.

The CPU returns to user mode and the C library wrapper returns the result to the application.

Parameters can be passed in CPU registers, in a block of memory whose address is passed in a register, or on the stack, depending on the processor ABI and system-call convention. Linux x86-64 normally uses registers for syscall number and arguments.

5.3 strace

```
strace observes the system calls made by a Linux process. Example: strace -o trace.txt -e trace=openat,read,write,close ./c

5.4 Alternative: stdio fopen(), fread() and fwrite() instead of direct system calls
```

The stdio library adds a user-space buffer. If a file contains n bytes and the library buffer is approximately 4096 bytes, it can reduce the number of underlying read/write system calls to roughly ceil(n/4096), plus open/close and possible metadata calls. With an application that issues a system call for every b-byte chunk, the number of calls is approximately ceil(n/b). For a 10,000-byte file, a 4096-byte buffer needs about three data reads plus one EOF read in the provided direct-read loop; stdio may make a similar small number of underlying calls, though its exact buffering is implementation-dependent.

The library buffer consumes O(B) additional user-space memory, where B is the chosen buffer size. Direct system calls give explicit control and can be useful for low-level I/O, while stdio buffering is simpler and often reduces expensive transitions. Complexity of copying n bytes remains O(n) time and O(B) auxiliary space.

QUESTION FROM THE PROVIDED ASSIGNMENT

System-call category | Purpose | Example in hospital

Process control | Create, execute, terminate and synchronise processes. | Start a database backup or clinical application; wait for completion.

File management | Create, open, read, write, close and manage file attributes. | Read configuration, write audit logs or access MRI files.

Device management | Request/release devices and perform device I/O. | Read monitor sensors or communicate with an imaging controller.

Information maintenance | Get/set time, system data, process and system attributes. | Timestamp alarms and query process/resource information.

Communication | Create communication channels and send/receive data. | Transfer records between hospital services.

Protection | Set permissions and control access. | Check access rights to a patient record.